Clustering

Lesson 1 - What is Clustering?

- A set of techniques used to partition data into groups, or clusters.
- Clusters are loosely defined as groups of data objects that are more similar to other objects in their cluster than they are to data objects in other clusters.

Clustering helps identify two qualities of data:

1. Meaningfulness
2. Usefulness

Meaningful: clusters expand domain knowledge.

Usefulness: clusters can serve as an intermediate step in a data pipeline.

Clustering is an unsupervised machine learning technique, meaning the data are not labelled with class. The results are heuristic meaning results are not reproducible.

Businesses use clustering for customer segmentation. The clustering results segment customers into groups with similar purchase histories, which businesses can then use to create targeted advertising campaigns. In the medical field, researchers applied clustering to gene expression experiments. The clustering results identified groups of patients who respond differently to medical treatments.

Lesson 2 - Why Clustering?

Why clustering?

- An insurance company wants to offer insurance to its customers.
- Let's say, they look at the details of each customer and based on this information, decide which insurance option should be given to which customer.
- Is it a sensible thing to look at the details of each customer separately and then make a decision?
- Segmentation of customers into different groups is one solution. For instance, the company can group the customers based on their income.
- The bank can now make different strategies or offers, one for each income group.

Popular applications of clustering:

- Retail – cluster the customers according to their buying patterns. Eventually recommending a suitable product.
- Banking – identify genuine customers. Fraud detention in loans etc.
- Insurance – mostly fraud detection and potential risky customers.

Lesson 3 – Segmentation

Segmentation is a technique to divide a population into logical groups that have a common need. Segments are created based on a plethora of variables.

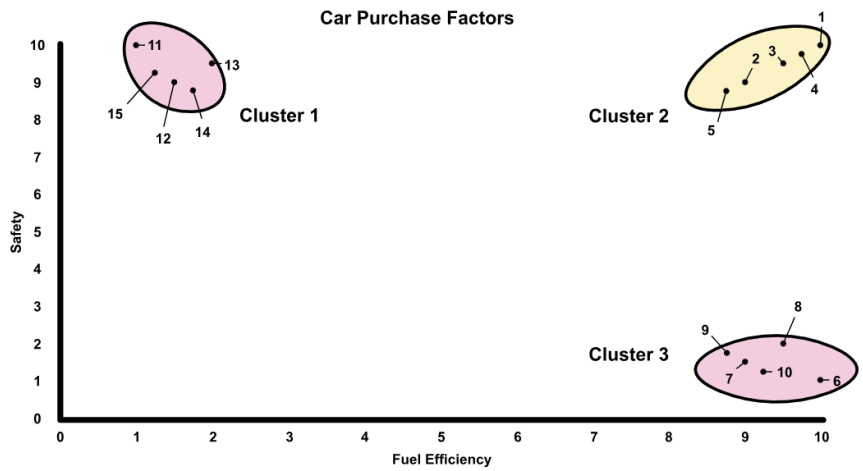
Main Variables:

Base - used in the segmentations for defining clusters	**Descriptor** - used to profile and define identified clusters

The following table shows a simple dataset with fifteen respondents and their response (1-10 scale) to car purchase factors.

Respondent	Safety	Fuel efficiency
1	10	10
2	9	9
3	9.5	9.5
4	9.75	9.75
5	8.75	8.75
6	10	1
7	9	1.5
8	9.5	2
9	9.75	1.75
10	8.75	1.25
11	1	10
12	1.5	9
13	2	9.5
14	1.75	8.75
15	1.25	9.25

Two factors, safety and fuel efficiency, are plotted here which shows three clusters of customers. For such simple and well-separated clusters, segmentation is easy to visualise but with millions of customers and not well-separated clusters, the segmentation becomes challenging.



Lesson 4 – Dis/similarity Between Data Objects

Similarity and dissimilarity are often measured in terms of distance between data objects.

This is accomplished using the following mathematical formulae:

1. Euclidean Distance
2. Simple Matching Coefficient (SMC)
3. Jaccard Coefficient
4. Cosine Similarity
5. Pearson Correlation Coefficient
6. Bregman Divergence

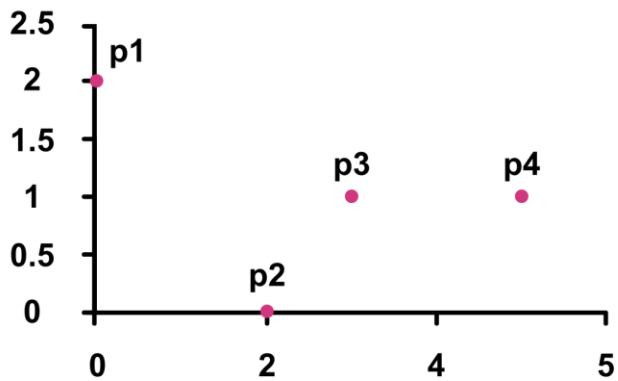
Lesson 5 – Euclidean Distance

Euclidean distance is used to calculate the linear distance between two points.

$$d(x,y) = \sqrt{\sum_{k=1}^{n} (x_k - y_k)^2}$$

$$d(p1, p2) = \sqrt{(0-2)^2 + (2-0)^2} = 2.8$$

In this example, the distance of points P1, P2, P3 and P4 are illustrated.



	p1	*p2*	*p3*	*p4*
p1	0.0	2.8	3.2	5.1
p2	2.8	0.0	1.4	3.2
p3	3.2	1.4	0.0	2.0
p4	5.1	3.2	2.0	0.0

Lesson 6 – Simple Matching Coefficient (SMC)

What if the data is binary and symmetric?

A binary symmetric variable is said to be symmetric if both of its states are of the same value and carry the same weight, and there is no preference as such as which outcome should be mapped to 0 or mapped to 1. For example, the attribute of gender (male 1, female 0).

Measuring the SMC for two customers based on 8 products that have (1)/ have not (0) purchased

customer 1 = (1, 0, 1, 1, 0, 1, 0, 1)

customer 2 = (1, 0, 1, 0, 1, 1, 0, 0)

- f_{11} = *both customers purchased*
- f_{10} = *first customer did purchase it but second customer did not*
- f_{01} = first customer did not purchase it but second customer did
- f_{00} = *neither customers purchased*

$$SMC = \frac{f_{01} + f_{10}}{f_{01} + f_{10} + f_{11} + f_{00}}$$

$$SMC = \frac{1+2}{1+2+3+2} = 0.375$$

		Object 2	
		1	0
Object 1	1	f_{11}	f_{10}
	0	f_{01}	f_{00}

Lesson 7 – Jaccard Coefficient

This is applicable for asymmetric data, where we only care for the presence of binary attributes. An asymmetric binary attribute is one in which outcomes are not of equal importance.

For example, with shopping data, we would be more concerned with similar items customers are buying.

$$Jaccard = \frac{f_{01} + f_{10}}{f_{01} + f_{10} + f_{11}}$$

Lesson 8 – Cosine Similarity

When the data is not binary, SMC and Jaccard measures cannot be used.

	Cow	Pig	Dog	Cat	Log	Bug	Fox	Ape	Man	Car
X	5	0	3	0	2	0	0	2	0	0
Y	3	0	2	0	1	1	0	1	0	1

$$\cos_sim(x, y) = \frac{x \times y}{\| x \| \times \| y \|} = \frac{5\times3+0\times0+3\times2.....0\times1}{\sqrt{(5\times5+0\times0...0\times0)} \times \sqrt{(3\times3+0\times0...1\times1)}} = \frac{25}{6.48\times4.12} = 0.94$$

‖ x ‖ means the norm of the element x of a normed vector space.

Lesson 9 – Types of Clusters

Well-separated: Any point in a cluster is closer to every other point in the cluster than any point not in the cluster

Centre-based: objects are closer to centre than to the centre of any other cluster. Centre can be centroid, or the most representative point of a cluster

Density-based: a dense region of data points, which is separated by low-density regions from other regions of high density.

Used when the clusters are irregular or intertwined and when noise and outliers are present

Concept-based: data share some common property or concept

Lesson 10 – Clustering Methods

There are many clustering methods, but we will be focusing on the first method only in this lecturecast.

1. **K-means - Partitioning Method**
2. Hierarchical clustering - Agglomerative Method
3. DBSCAN: Density Based Spatial Clustering of Applications with Noise – Density Based Method

Lesson 11 – K-Means Clustering

The Basic Ideas behind K-Means Clustering are:

- Assume a fixed number of clusters, *K*
- And the Goal? To create “compact” clusters

K-Means clustering requires a few steps:

1. First step is to randomly select k centroids, where k is equal to the number of clusters pre-set by the user.
2. Centroids are data points representing the centre of a cluster.
3. K-Means works through a two-step approach called expectation-maximisation.
4. Expectation step assigns each data point to its nearest centroid.
5. Maximisation step computes the mean of all the points for each cluster and sets the new centroid.

K-Means is made of aforementioned steps which in pseudo code terms can be written as follows:

Algorithm 1 *k*-means algorithm

```
Specify the number k of clusters to assign.
Randomly initialize k centroids.
repeat
Expectation: Assign each point to its closest centroid.
Maximization: Compute the new centroid (mean) of each cluster.
until The centroid positions do not change.
```

Sum of squared error (SSE) is used to determine the quality of K-Means.

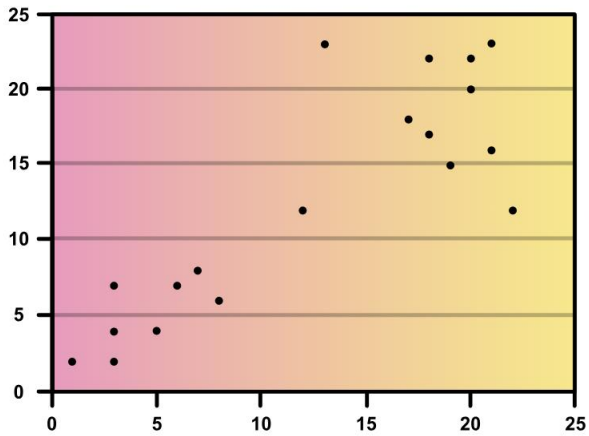
Strengths and Weaknesses

1. Strenghts:
- It is a simple iterative method.
- A user provides a value for "K" (the number of clusters).

2. Weaknesses:
- The algorithm is too simple and therefore may sometimes yield bad results.
- It can be difficult to guess the correct "K".

Example

In this example, we will find clusters (K = 2).



2	2
3	2
5	4
7	8
3	7
22	12
12	12
13	23
20	20
19	15
21	23
20	22
18	22
17	18
8	6
6	7
21	16
18	17
3	4
1	2

First Iteration

1.

2	2
3	2
5	4
7	8
3	7
22	12
12	12
13	23
20	20
19	15
21	23
20	22
18	22
17	18
8	6
6	7
21	16
18	17
3	4
1	2

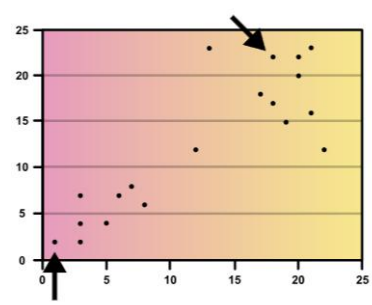
Original Data

2.

Original Data

	x	y
C1	2	2
	3	2
	5	4
	7	8
	3	7
	22	12
	12	12
	13	23
	20	20
	19	15
C2	21	23
	20	22
	18	22
	17	18
	8	6
	6	7
	21	16
	18	17
	3	4
	1	2

Select initial centroids



3.

	x	y
C1	2	2
	3	2
	5	4
	7	8
	3	7
	22	12
	12	12
	13	23
	20	20
	19	15
C2	21	23
	20	22
	18	22
	17	18
	8	6
	6	7
	21	16
	18	17
	3	4
	1	2

Original Data

Calculate "closeness" using Euclidean distance

x	y	To C1	To C2
2	2	0	28.32
3	2	1	27.66
5	4	3.61	24.84
7	8	7.81	20.52
3	7	5.1	24.08
22	12	22.36	11.05
12	12	14.14	14.21
13	23	23.71	8
20	20	25.46	3.16
19	15	21.4	8.25
21	23	28.32	0
20	22	26.91	1.41
18	22	25.61	3.16
17	18	21.93	6.4
8	6	7.21	21.4
6	7	6.4	21.93
21	16	23.6	7
18	17	21.93	6.71
3	4	2.24	26.17
1	2	1	29

Closeness to centroids

4.

	x	y
C1	2	2
	3	2
	5	4
	7	8
	3	7
	22	12
	12	12
	13	23
	20	20
	19	15
C2	21	23
	20	22
	18	22
	17	18
	8	6
	6	7
	21	16
	18	17
	3	4
	1	2

Original Data

Calculate "closeness" using Euclidean distance

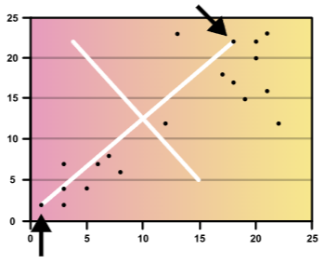
x	y	To C1	To C2
2	2	0	28.32
3	2	1	27.66
5	4	3.61	24.84
7	8	7.81	20.52
3	7	5.1	24.08
22	12	22.36	11.05
12	12	14.14	14.21
13	23	23.71	8
20	20	25.46	3.16
19	15	21.4	8.25
21	23	28.32	0
20	22	26.91	1.41
18	22	25.61	3.16
17	18	21.93	6.4
8	6	7.21	21.4
6	7	6.4	21.93
21	16	23.6	7
18	17	21.93	6.71
3	4	2.24	26.17
1	2	1	29

Closeness to centroids

5.

	x	y
C1	2	2
	3	2
	5	4
	7	8
	3	7
	22	12
	12	12
	13	23
	20	20
	19	15
C2	21	23
	20	22
	18	22
	17	18
	8	6
	6	7
	21	16
	18	17
	3	4
	1	2

Original Data



x	y	To C1	To C2
2	2	0	28.32
3	2	1	27.66
5	4	3.61	24.84
7	8	7.81	20.52
3	7	5.1	24.08
22	12	22.36	11.05
12	12	14.14	14.21
13	23	23.71	8
20	20	25.46	3.16
19	15	21.4	8.25
21	23	28.32	0
20	22	26.91	1.41
18	22	25.61	3.16
17	18	21.93	6.4
8	6	7.21	21.4
6	7	6.4	21.93
21	16	23.6	7
18	17	21.93	6.71
3	4	2.24	26.17
1	2	1	29

Closeness to centroids

6.

C1 = ((2+3+5+7+3+12+8+6+3+1)/10,

(2+2+4+8+7+12+6+7+4+2)/10)

= (5,5.4)

C2 = ((22+13+20+19+21+20+18+17+21+18)/10,

(12+23+20+15+23+22+22+18+16+17)/10)

= (18.9,18.8)

Since the new centroids are different from the previous iteration, computation must continue, and these two points will be used to calculate “closeness" using Euclidean distance.

Centroids will not change, so computation ends.

The final clusters are

Cluster 1 = {(2,2),(3,2),(5,4),(7,8),(3,7),(12,12),(8,6),(6,7),

(3,4),(1,2)}

Cluster 2 = {(22,12),(13,23),(20,20),(19,15),(21,23),(20,22),

(18,22),(17,18),(21,16),(18,17)}

Second Iteration

x	y	To C1	To C2
2	2	0	28.32
3	2	1	27.66
5	4	3.61	24.84
7	8	7.81	20.52
3	7	5.1	24.08
22	12	22.36	11.05
12	12	14.14	14.21
13	23	23.71	8
20	20	25.46	3.16
19	15	21.4	8.25
21	23	28.32	0
20	22	26.91	1.41
18	22	25.61	3.16
17	18	21.93	6.4
8	6	7.21	21.4
6	7	6.4	21.93
21	16	23.6	7
18	17	21.93	6.71
3	4	2.24	26.17
1	2	1	29

Result of First Iteration

x	y	To C1	To C2
2	2	4.53	23.83
3	2	3.94	23.13
5	4	1.4	20.3
7	8	3.28	16.07
3	7	2.56	19.8
22	12	18.24	7.47
12	12	9.62	9.69
13	23	19.33	7.24
20	20	20.93	1.63
19	15	16.98	3.8
21	23	23.79	4.7
20	22	22.37	3.38
18	22	21.08	3.32
17	18	17.4	2.06
8	6	3.06	16.81
6	7	1.89	17.48
21	16	19.19	3.5
18	17	17.42	2.01
3	4	2.44	21.72
1	2	5.25	24.55

Result of Second Iteration

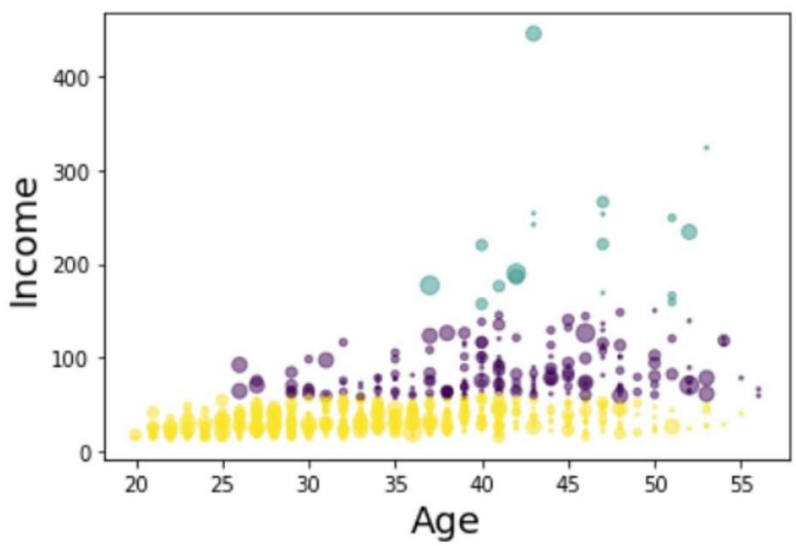
Centroids will not change, so computation ends.

The final clusters are

Cluster 1 = {(2,2),(3,2),(5,4),(7,8),(3,7),(12,12),(8,6),(6,7),
(3,4),(1,2)}

Cluster 2 = {(22,12),(13,23),(20,20),(19,15),(21,23),(20,22),
(18,22),(17,18),(21,16),(18,17)}

```
area = np.pi * ( X[:, 1])**2
plt.scatter(X[:, 0], X[:, 3], s=area, c=labels.astype(np.float64), alpha=0.5)
plt.xlabel('Age', fontsize=18)
plt.ylabel('Income', fontsize=18)
plt.show()
```



Lesson 12 – Optimising K in K-Means

One disadvantage of K-Means compared to more advanced clustering algorithms is that the algorithm must be told how many clusters, *k*, it should try to find. To optimize *k* we cluster a fraction of the data for different choices of k and look for an "elbow" in the cost function.

Lesson 13 – Choosing Appropriate K Value

1. The elbow method
2. The silhouette coefficient

These methods are often used complementarily.

Lesson 14 – Evaluating Clusters by Sum of Squared Error (SSE)

For each point, the error is the distance to the nearest centroid or representative point.

To get SSE, we square these errors and sum them.

$$SSE = \sum_{i=1}^{K} \sum_{x \in C_i} dist^2(m_i, x)$$

1. X is a data point in cluster Ci and mi is the representative point for cluster Ci
2. Reduce SSE by increase K
3. Good clustering – small K, reduced SSE

Lesson 15 – The Elbow Method

1. Run K-Means multiple times, increment/change K value on each iteration
2. Record the SSE

This example show the elbow method, looking at K = 3

```
sse = []
for k in range(1, 11):
kmeans = KMeans (n_clusters=k, init= "random", n_init= 10, max_iter= 300)
kmeans.fit(desc_matrix)
sse.append(kmeans. inertia_)
plt.style.use("fivethirtyeight")
plt.plot(range(1, 11), sse)
plt.xticks(range(1, 11))
plt.xlabel("Number of clusters")
plt.ylabel("SSE")
plt.show()
from kneed import Knee Locator
kl KneeLocator (
range(1, 11), sse, curve="convex", direction="decreasing"
)
kl.elbow
```

This is the output elbow plot: Elbow metric. k = 3; x-axis = Number of clusters; y-axis = SSE.

Lesson 16 – Will K-Means Always Converge?

• K-means will converge for common similarity measures mentioned above

• Finite number of potential centres

• SSE always becomes smaller

• Most of the convergence happens in the first few iterations

• Often the stopping condition is changed to 'Until relatively few points change clusters'

Complexity is:

• O = Big O notation and is the relative representation of the complexity of an algorithm.

• n = number of points

• K = number of clusters

• I = number of iterations

• d = number of attributes

Lesson 17 – K-Means Accuracy

Accuracy in K-Means can be determined using the average distance between data points within a cluster. In addition, the average distance of data points from their cluster centroids can also be used as a metric of error for the clustering algorithm

Lesson 18 – Cluster Evaluation

1. Determine cluster tendency
2. Determine the correct number of clusters
3. Evaluate how well clusters fit data
4. Compare clusters to externally known results

5. Compare two sets of clusters to determine which one is better

Lesson 19 – Cohesion and Separation

Cohesion (measures cluster's homogeneity)

$$cohesion = \sum_{x \in C, y \in C} proximity(x, y)$$

Separation (how well separated the clusters are)

$$separation = \sum_{x \in C_i, y \in C_j} proximity(x, y)$$

A good clustering produces clusters with low cohesion value (data points within the clusters are close together or homogeneous), at the same time inter cluster separation is high (well separated).

Example

Assume that we have the following data and two clustering solutions.

p1	40	53
p2	22	38
p3	35	32
p4	26	19
p5	8	41
p6	45	30

Two solutions

S1 = {(p1,p2,p3), (p4,p5,p6)}

S2 = {(p3,p6,p4), (p1,p2,p5)}

	p1	p2	p3	p4	p5	p6
p1	0	24	22	37	34	23
p2	24	0	15	20	14	25
p3	22	15	0	15	28	11
p4	37	20	15	0	29	22
p5	34	14	28	29	0	39
p6	23	25	11	22	39	0

Cohesion

Cohesion(S1) = d(p1,p2) + d(p1,p3) + d(p2,p3) + d(p4,p5) + d(p4,p6) + d(p5,p6) = 171

Cohesion(S2) = d(p3,p6) + d(p3,p4) + d(p6,p4) + d(p1,p2) + d(p1,p5) + d(p2,p5) = 120

Lesson 20 – Silhouette Method

A measure of cluster cohesion and separation.

It quantifies how well a data point fits into its assigned cluster based on two factors:

1. How close the data point is to other points in the cluster.
2. How far away the data point is from points in other clusters.

Plotting the average silhouette scores for each *k* shows that the best choice for *k* is 2/3 since it has the maximum score. Ultimately, your decision on the number of clusters to use should be guided by a combination of domain knowledge and clustering evaluation metrics.

```
from sklearn.metrics import silhouette_score, silhouette_samples
silhouette_coefficients = []
# Notice you start at 2 clusters for silhouette coefficient
for k in range(2, 10):
kmeans = KMeans (n_clusters=k, init= "k-means++"; ',n_init= 12, max_iter= 300)
kmeans.fit(X)
score = silhouette_score (X, kmeans.labels_) silhouette_coefficients.append(score)
silhouette_coefficients
[0.6857530340991117,
0.5622416593374175,
0.4779880648501076,
0.3608818966276197,
0.3572196642812843,
0.3196751106476776,
0.2724703991045969,
0.2678878628937257]
```

Lesson 21 – Hierarchical Clustering

1. **Agglomerative Clustering**
   Start with each data item as a cluster, and then iteratively merge the closest pair of clusters.
2. **Divisive Clustering**
   Start with one, all-inclusive cluster, and then iteratively split a cluster until only singleton cluster remain.

References

- Seif, G. (2018) The 5 Clustering Algorithms Data Scientists Need to Know. Available from: https://towardsdatascience.com/the-5-clustering-algorithms-data-scientists-need-to-know-a36d136ef68 [Accessed on 16 May 2022]

- Bonthu, H. (2021) Understanding KMeans Clustering for Data Science Beginners. Available from: https://www.analyticsvidhya.com/blog/2021/08/kmeans-clustering/ [Accessed on 16 May 2022

- Grootendrorst, M. (2019) Cluster Analysis: Create, Visualize and Interpret Customer Segments. Available from: https://towardsdatascience.com/cluster-analysis-create-visualize-and-interpret-customer-segments-474e55d00ebb [Accessed on 16 May 2022]

- Thinsungnoen, T. et al. (2015) The Clustering Validity with Silhouette and Sum of Squared Errors. Proceedings of the 3rd International Conference on Industrial Application Engineering. DOI: 10.12792/iciae2015.012